1] K-means clustering

It is an unsupervised learning algorithm used to group data into k clusters based on similarity (distance).

Steps :

- Choose the no. of clusters (k)
- Find similar data by using euclidean distance formula
- Each cluster has a centroid (mean point)
- Assigns points to nearest centroid
- Iteratively updates centroids
- Stops when centroids no longer change

Mathematical Implementation :

Step 1 : Initialize centroids

choose k initial centroids randomly :

$$C_1, C_2, C_3, \ldots, C_k$$

Step 2 : Distance calculation : compute distance b/w each data point & centroid

$$d = \sqrt{\Sigma (x_i - c_i)^2}$$

Step 3 : Assign clusters : Assign each point to nearest centroid

Step 4 : Update centroids : $C = \frac{1}{n} \Sigma x_i$

New centroid = mean of all points in that cluster

Step 5 : Repeat steps 2-4 until centroids do not change

Example:

Point	X	Y
A	1	1
B	2	2
C	5	5
D	6	6

Choose $k = 2$, Initial centroids, $C_1 = (1,1)$
$C_2 = (5,5)$

Step 1: Assign points to clusters

Distance from A (1,1): To $C_1 = 0$
To $C_2 \approx 5.65 \rightarrow$ assign C_1

Distance from B (2,2): To $C_1 = 1.41$
To $C_2 = 4.24 \rightarrow C1$

Distance from C (5,5): To $C_1 = 7.07$
$C_2 = 1.41 \rightarrow C2$

Step 2: Update Centroids

Cluster 1 (A, B): $C_1 = \left(\frac{1+2}{2}, \frac{1+2}{2}\right) = (1.5, 1.5)$

Cluster 2 (C, D): $C_2 = \left(\frac{5+6}{2}, \frac{5+6}{2}\right) = (5.5, 5.5)$

Step 3: Reassign points

- Points remain in same clusters

No change

Characteristics :- Fast, easy to understand
to find cluster (k) we can use WCSS (elbow method)

2] Hierarchical clustering :

It is an unsupervised learning algorithm that groups data into clusters by creating a tree-like structure (dendogram) based on similarity.

Types : (1) Agglomerative (Bottom-Up)
(2) Divisive (Top-Down)

Working :

Step 1 : Start with individual clusters
Each data point is its own cluster

Step 2 : Compute Distance matrix
Find distance b/w all pairs of points

Step 3 : Merge closest clusters

Step 4 : Update distance matrix
Recalculate distances after merging

Step 5 : Repeat merging until only one cluster remain

Step 6 : Cut the dendogram at a certain level
Get required number of clusters

Distance, $d = \sqrt{\Sigma(x_i - y_i)^2}$

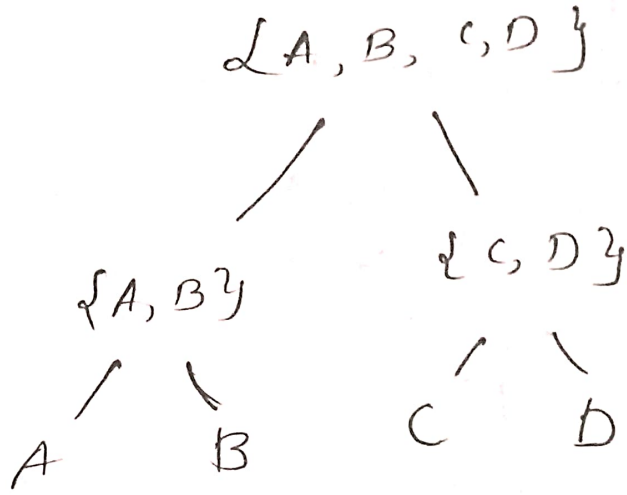
Example : Data Points : A(1), B(2), C(5), D(6)

Step1 : {A}, {B}, {C}, {D}

Step2 : A - B = 1
C - D = 1
Others larger

Step3 : Merge {A, B}
Merge {C, D}

Step4 : Merge again
{A, B, C, D}



Adv :

- No need to choose k initially
- Easy to visualize
- works well for small datasets

3] DBSCAN [Density-Based Spatial Clustering of Applications with Noise]

It is an unsupervised learning algorithm used for clustering that groups data points based on density, & identifies outliers (noise)

Parameters : ϵ (Epsilon) : Radius around a point

MinPts : Min no. of points req to form a dense region

Mathematical implementation :

Step 1 : Distance calculation, $d = \sqrt{\sum (x_i - y_i)^2}$

Step 2 : Neighborhood definition, $N_\epsilon(p) = \{q \mid d(p,q) \leq \epsilon\}$

All points within radius ϵ

Step 3 : Core point condition, $|N_\epsilon(p)| \geq$ Min Pts

Step 4 : Cluster formation :- strat from a core point
- Add all density-reachable points
- Expand cluster recursively

How it works :

1) Pick an unvisited point
2) Find its ϵ-neighborhood
3) If it is a ~~cluster~~ core point → start a new cluster
4) Expand cluster by adding neighbors
5) If not → mark as noise
6) Repeat until all points are processed

Example: Given points : A, B, C, D, E

Assume : ϵ = small radius, Min Pts = 2

Step 1 : Identify core points

- A, B, C → close together → core points
- D → near cluster → border
- E → isolated → noise

Step 2 : Form cluster

Cluster 1 = {A, B, C, D}

Noise = {E}

Importance :

- No need to specify k
- Detect outliers
- Handles arbitary shapes
- Robust to noise
- Real-word applin : - GPS / location clustering
  - Image segmentation
  - customer behavior analysis
  - Fraud detection

A] Gaussian Mixture Model (GMM):

It is an unsupervised learning algorithm used for clustering, which assumes the data is generated from a mixture of multiple Gaussian (normal) distributions.

Mathematical implementation:

Gaussian Distribution formula: $P(x) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(x-\mu)^2}{2\sigma^2}\right)$

GMM probability: $P(x) = \sum_{k=1}^{K} \pi_k \cdot N(x \mid \mu_k, \sigma_k^2)$

where k = no of clusters
π_k = no of weights
N = Gaussian distribution

How it works:

Step 1: Initialize: Choose no of clusters K
Initialize μ, σ, π randomly

Step 2: Expectation step (E-step)
$P(\text{Cluster}_k \mid x)$

Step 3: Maximization step (M-step)
Update parameters: Mean (μ), Variance (σ^2)
Mixing coeff (π)

Step 4: Repeat E-step & M-step until convergence

Example : Data points : 1, 2, 3, 8, 9, 10

Assume $k = 2$

Step 1 : Initialize

- Cluster 1 → around 2
- Cluster 2 → around 9

Step 2 : E - step

- Points 1, 2, 3 → high probability for C1
- Points 8, 9, 10 → high prob for C2

Step 3 : M - step - update means : $\mu_1 \approx 2$

$\mu_2 \approx 9$

Step 4 : Repeat : Probabilities refine

Clusters stabilize

5] Evaluation metrics used for clustering algorithm

(1) <u>Silhouette</u> :

- Evaluates quality of clusters
- if silhouette score close to +1 → good cluster
- around 0 → clusters overlap
- close to -1 → poor cluster

(2) <u>Davies bouldin</u>: calculates avg similarity measure to each cluster

0 to 0.5 → good cluster
0.5 to 1 → reasonable
> 1 → clusters may be overlapping

(3) <u>Calinski harabiz</u> : Measure how similar an object is to its own cluster

↑ score → better model